

Course: CSE 306 – Network Programming
LAB 2 – Python

Student name	<i>Phan Ngoc Hanh Nhi</i>
Student ID	<i>2131209002</i>

The requirement is to build a Python project for network configuration management, used to store and manage information about network nodes (clients/servers).

REQUIREMENT 1 – WORKING ENVIRONMENT (MANDATORY)

- Create a virtual environment using venv or pipenv
- Activate the virtual environment before running the program
- Install at least one external library (black, requests)
- Export the requirements.txt file

! Do not use Python global

Question: Explain the difference between Python global and Python virtual environments. Why is it necessary to use them?

REQUIREMENT 2 – PROJECT STRUCTURE

The project must be organized according to the following structure:

```
lab2_your_computer_ID/  
├── .venv/  
├── app/  
│   ├── models/  
│   │   └── node.py  
│   ├── utils/  
│   │   └── validator.py  
│   └── config/  
│       └── config_manager.py  
├── data/  
│   └── nodes.json  
└── main.py
```

REQUIREMENT 3 – DATA MODEL (OOP)

Create a **NetworkNode** class (node.py) with the following information:

- node_id (automatically generated)
- ip
- port
- role (client or server)
- protocol (TCP or UDP)

Constraints:

- The IP address must be valid.
- The port must be between 1 and 65535.
- The role must only accept either client or server.
- Invalid data will result in an error (Exception).

REQUIREMENT 4 – DATA VALIDATION

Write a separate module (validator.py) to check:

- IP (valid IPv4 address)
- Port
- Role
- Protocol (only TCP or UDP)
- Do not print errors to the screen
- Errors must be handled using Exceptions

REQUIREMENT 5 – CONFIGURATION FILE MANAGEMENT

- (config_manager.py) Save the list of nodes to the file **data/nodes.json**
- Read node data from the file
- Support:
 - Add nodes
 - Delete nodes by node_id
 - Display the list of nodes

REQUIREMENT 6 – MAIN PROGRAM (CLI)

When the program runs (main.py), the following menus appear:

1. Add node
2. List nodes
3. Remove node
4. Exit

Constraints:

- Testing with port number is the last four digits of your ID.

```
1. Add node
2. List nodes
3. Remove node
4. Exit
Option: 1
IP: 127.0.0.1
Port: 0016
Role (client/server): client
Protocol (tcp/udp): tcp
Node added.
```

```
data > {} nodes.json > ...
1  [
2    {
3      "node_id": "4086875c-cd1d-4ab9-ae08-aaa53991eccc",
4      "ip": "127.0.0.1",
5      "port": 16,
6      "role": "client",
7      "protocol": "tcp"
8    }
9  ]
```